Standardization, Interoperability & Information Modelling

Session 1 — what it means to define an information standard, and looking at openADR through that lens

Joep van Genuchten, uuidea

November 10, 2026

uuid ea

 ©uuidea cc by-sa 4.0

whoami



- **Name**: Joep van Genuchten
- **Position/Role**: Data Architect/Contractor
- **Institution/Organization**: uuidea
- **Interests**:
  - Interoperability
  - Data and Semantics
  - Aligning different architectural layers
  - Taking architecture beyond the design phase
  - Open Source

uuid ea

©uuidea cc by-sa 4.0

Overview

- **1 — the foundations**
  - interoperability: what is actually involved
  - SGAM: where openADR sits — in a hands-on exercise
  - the Ship of Theseus: identity, meaning & MDM
  - information modelling 101: L0–L3, RDF, SKOS, OWL, SHACL
  - the requirements — the yardstick the rest of the day is built on
- **2 — the requirements, addressed one at a time**
  - Req. 1: kinds of information models — vocabulary vs. constraint
  - Req. 2: MOF — metamodels, expressivity, transformation
  - meaning: OWA/CWA, the open world
  - Req. 3: ontology alignment, upper ontologies
- **3 — in practice, and the lens turned back on openADR**
  - profiles: use-case specific models
  - the toolchain: Avro, JSON Schema, SQL — *generated*
  - Netbeheer Nederland: the practice
  - the three requirements, applied to openADR 3.0

uuid ea

©uuidea cc by-sa 4.0

Glossary — the words we'll use today (1/2)

One table, so everyone means the same thing by the same word

Term	What it is
Interoperability	A *community* quality: systems able to work together *without modification*.
Standard	A reference point against which other things can be evaluated or compared
Model	Something that a copy can be based on because it is an extremely good example of its type
Information model (IM)	The *model* of a domain's information. abbreviated *IM*.
Business Terms	The natutal language definitions of the things we talk about.
Vocabulary Model	information model that relies on set theoretical semantics to describe a domain. Built on the *open* world; reusable across use cases
Constraint model	The *data structure* of a use case: what fields exist and how much of each; built on the *closed* world.
Profile	A reference to an information model *plus* added constraints (scope, cardinality, value-space).
Serialization	A concrete *exchange format* for a specific pattern: Avro, JSON Schema, SQL DDL. Also called *schema* or *artifact* (L3).
Meta-model	A model *of* models (UML, OWL, SHACL, JSON Schema...).
Meta-meta-model	A model of meta-models (MOF, RDF/S).
Pivot model	A chosen meta-model with enough expressivity to transform to and from the others.

uuid ea

©uuidea cc by-sa 4.0

Glossary — the words we'll use today (2/2)

One table, so everyone means the same thing by the same word

Term	What it is
Ontology	A formal, machine-interpretable account of meaning and relationships (ISA, disjointness) — the terms other standards can align to.
Upper ontology	A high-level classification of reality (BFO, nen-2660) used as a common yardstick to spot semantic conflicts.
Alignment / mapping	A documented, reviewable link between two models' terms (`owl:equivalentClass`, `skos:closeMatch`).
SGAM	Smart Grid Architecture Model: Business / Function / Information / Communication / Component layers.
CIM / CGMES	IEC 61970 CIM is the energy *vocabulary*; CGMES is the ENTSO-E profile on top of it.
mRID	A globally unique, persistent identifier (IEC 61970-552) that survives change.

uuid ea

©uuidea cc by-sa 4.0

Interoperability — what!?

Definition 1

Merriam Webster *in·ter·op·er·a·bil·i·ty* — ability of a system (such as a weapons system) to work with or use the parts or equipment of another system.

Definition 2

Cambridge Dictionary Interoperability: the degree to which two products, programs, etc. can be used together, or the quality of being able to be used together.

As a data architect, I love me some well-defined terms!

uuid ea

©uuidea cc by-sa 4.0

Interoperability — what!? (2)

Definition 3

U.S. Department of Energy Interoperability: the ability of systems, personnel, or organizations to provide services to and accept services from other systems, personnel, or organizations, and to use the services so exchanged so that these organizations can operate together effectively.

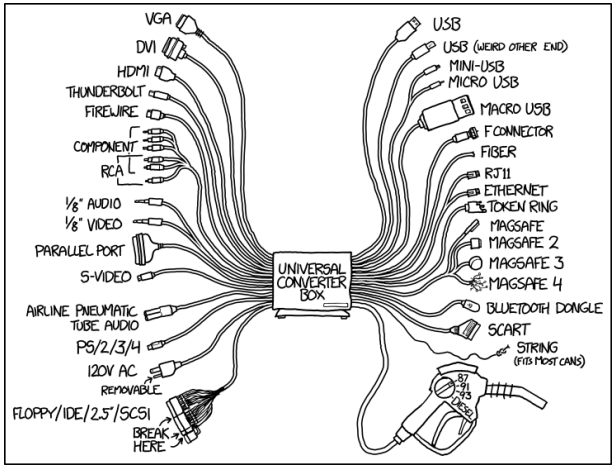
Definition 4

Red Hat Interoperability: the connection of data, applications, APIs, and devices across your IT organization to be more efficient, productive, and agile.

uuid ea

©uuidea cc by-sa 4.0

Interoperability — what!?



Wait... what!? This is interoperability?

uuid ea

 ©uuidea cc by-sa 4.0

Interoperability — what!?





uuid ea

Interoperability — what!?





Upon further reflection:

When systems can work together but have been *modified* to do so, they are integrated.
Merriam Webster: [systems that are] designed to work with another device or system *without modification* [are compatible].

©uuidea cc by-sa 4.0

Interoperability — what!?

EU legislation

Requires Apple (and all other smartphone manufacturers) to use **USB-C** for the data/charging interface. Motivation: the need for interoperability to reduce e-waste and increase customer choice of vendors.

Definition 5

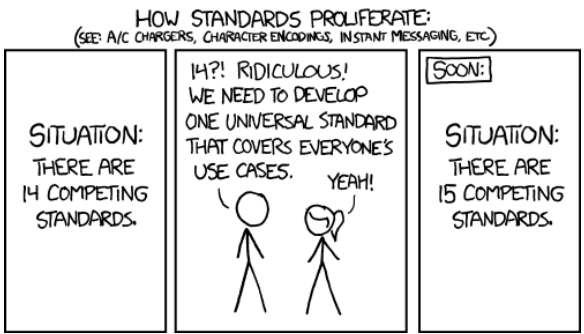
Compatible Designed to work with another device or system *without modification* (Merriam Webster).

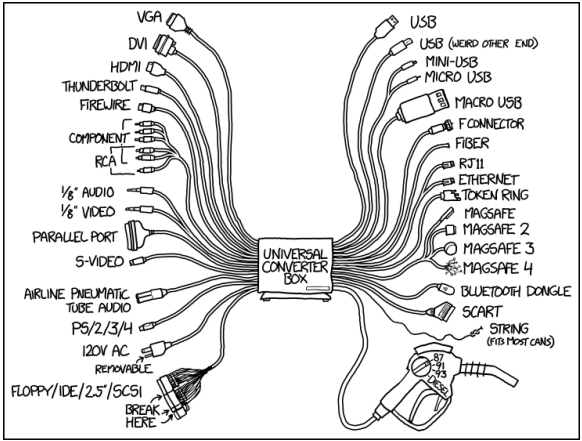
We need standards!

uuid ea

©uuidea cc by-sa 4.0

Interoperability — what!?





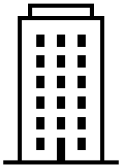
All of these are standards... so that is not enough either.

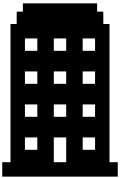
uuid ea

The importance of communities in the pursuit of interoperability

"I am sooo interoperable"

"No! I am sooo interoperable"





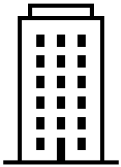
Two organizations — each claiming interoperability, with no one to interoperate with.

uuid ea

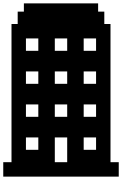
©uuidea cc by-sa 4.0

The importance of communities in the pursuit of interoperability

"We are sooo interoperable"



"We are sooo interoperable"

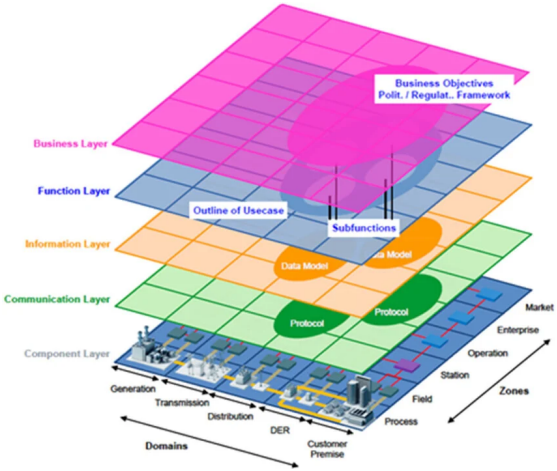


Interoperability is a quality of a community of contributors.

uuid ea

©uuidea cc by-sa 4.0

SGAM Refresher



uuid ea

©uuidea cc by-sa 4.0

SGAM: the layers

Layer	Question it answers
Business	Why do we exist? What capabilities do we have?
Function	How do we achieve our capabilities, what is the process?
Information	What information is exchanged for the purpose of the process? *mean*?
Communication	How do we transmit it? Integration patterns, protocols, etc
Component	What physical/digital artifacts run it? Software, hardware, platforms.

Key point

Interoperability failures usually happen*cross-layer*: you agree on the protocol, but not on what the bytes *mean* — or on what the business process requires.

uuid ea

©uuidea cc by-sa 4.0

Exercise: plot openADR on the SGAM framework

Working exercise — ~20 minutes, in pairs

1. Take the openADR building blocks you know best:
2. Place each of them on the domains and zones
3. Place each of them on the SGAM layers: Business | Function | Information | Communication | Component

Deliverable

Draw out the zones/dimensions grid and plot your building block on the grid and describe each layer in terms of your block/usecase

uuid ea

©uuidea cc by-sa 4.0

The Ship of Theseus

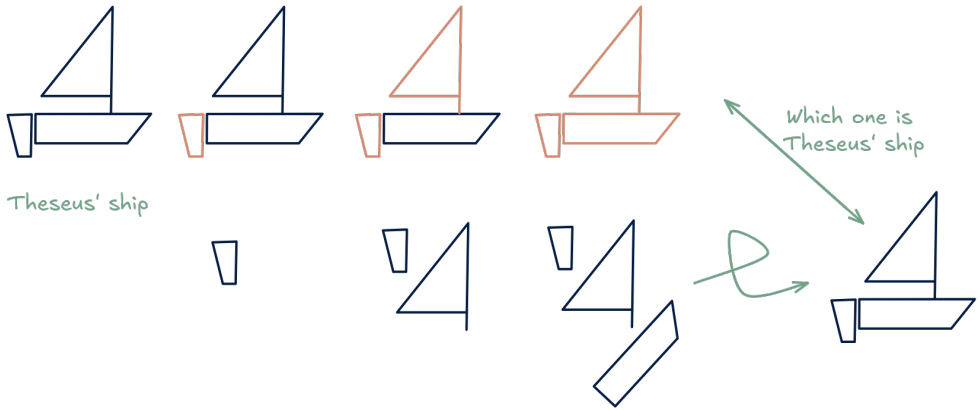


Figure: Planks replaced one by one, spare parts assembled into a second ship.

uuid
ea

©uuidea cc by-sa 4.0

The Ship of Theseus

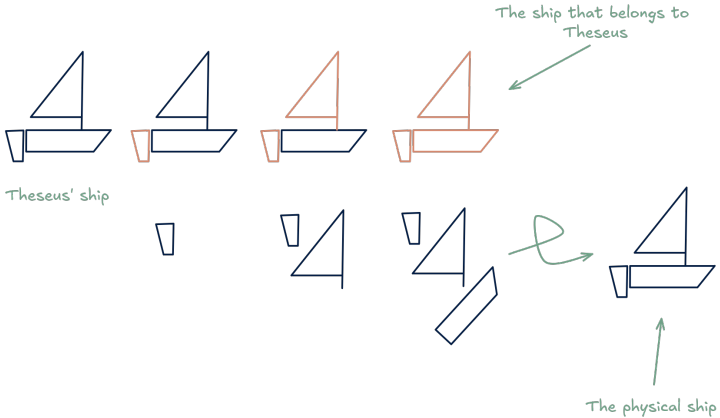


Figure: Two perspectives: the ship that *belongs* to Theseus, and the *physical* ship made of the original planks.

©uuidea cc by-sa 4.0

The Ship of Theseus

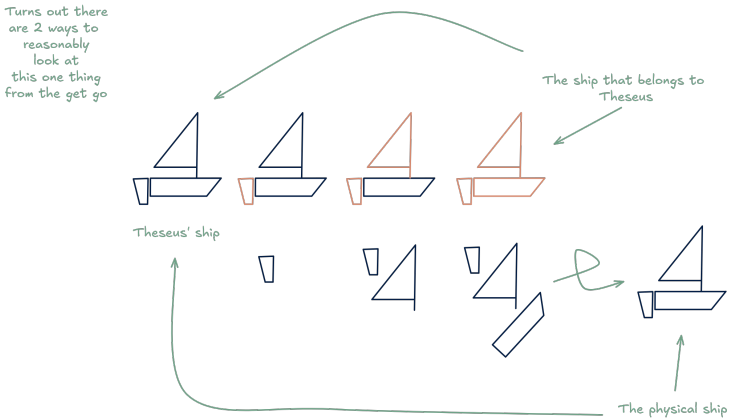


Figure: Turns out there are two (or more) ways to reasonably look at this one thing — from the get-go.

uuid ea

©uuidea cc by-sa 4.0

Market services add an extra layer of interpretation

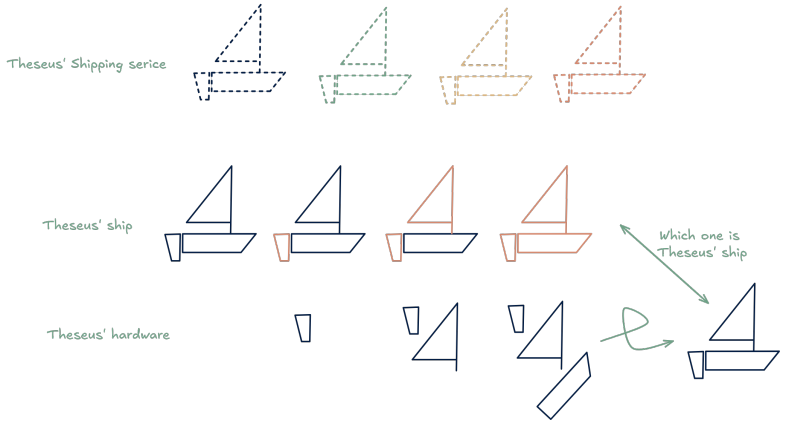


Figure: Three levels of the ship of *Theseus*: the shipping service, the ship, and the hardware — and still the question "which one is Theseus' ship?"

uuid ea

©uuidea cc by-sa 4.0

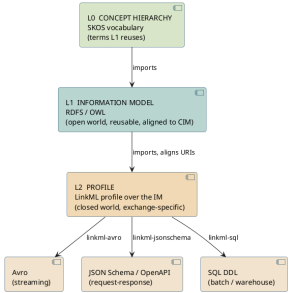
Master data & meaning: what actually has to be stable?

The ship, the shipping service, the hardware... which one are we talking about when we refer to *the ship*? and if that is unclear, what does 'ship' actually *mean*?

uuid
ea

©uuidea cc by-sa 4.0

The layer model (L0–L3)



L0 Concept hierarchy
SKOS: the terms L1 reuses

L1 Information Model
open world, reusable, normative definitions

L2 Profile
closed world, *adds constraints*

L3 Serializations
schema of a specific exchange pattern

L4 Instance data
record, row, JSON doc — *mostly out of scope*

Direction of authority: L0 < L1 < L2 ⊢ L3.
Never edit an artifact; edit the model above it and regenerate.

uuid ea

©uuidea cc by-sa 4.0

Where we stand: NORA/MIM + the W3C stack

MIM (*Metamodel voor Informatiemodellen*) — NORA's Informatielaag — splits modelling into five types; in this deck they are L0–L4, and we cover L0–L3.

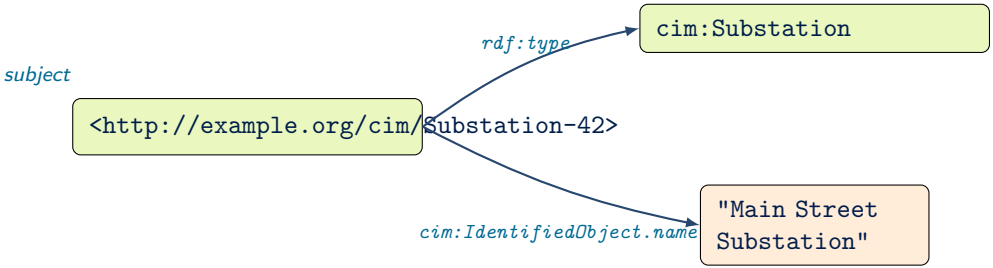
MIM type	**W3C / formalism**	**In this deck**
Model van begrippen	**SKOS** vocabulaire	**L0** Concept hierarchy
Conceptueel informatiemodel	**RDFS** + **OWL**	**L1** Information Model
Logisch informatiemodel	**SHACL** constraints	**L2** Profile
Fysiek / technisch gegevensmodel	**schema**: Avro, JSON-Schema, SQL	**L3** Serializations
Registraties	instance = record, row, JSON doc	**L4** runtime data — *mostly out of scope*

The *model van begrippen* is our L0 — a *SKOS* vocabulary: *"een vocabulaire is geen informatiemodel"* — terms we *reuse*, not design.

©uuidea cc by-sa 4.0

RDF — the triple

Everything is `subject predicate object`



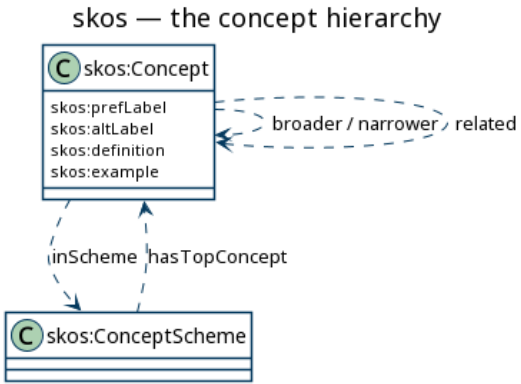
Same `subject` in both triples. The first object is a *resource* (`cim:Substation`); the second is a *literal* (a string).

uuid ea

©uuidea cc by-sa 4.0

SKOS — the concept hierarchy (L0)

Concepts, relations, and how they relate



Concepts

- `skos:Concept` — a single idea in a vocabulary
- `skos:ConceptScheme` — the set that groups them

Relations (concept → concept)

- `broader` / `narrower` — inverse hierarchy pair
- `related` — associative, non-hierarchical
- plus transitive variants (`...Transitive`)

Attributes & membership

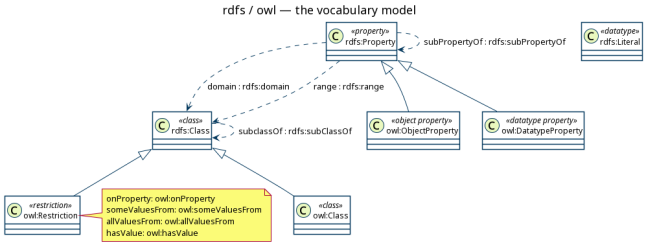
- `prefLabel` / `altLabel` — display terms
- `definition` / `example` — meaning
- `inScheme` / `hasTopConcept`

A vocabulary is not an *information model* — it is the **L0** layer: the named concepts we *reuse* to describe L1.

©uuidea cc by-sa 4.0

RDFS / OWL — the vocabulary model (L1)

Classes, properties, and how they relate



Classes

- `rdfs:Class`, `owl:Class` — *sets* of resources
- `owl:Restriction` — anonymous classes

Properties

- `owl:ObjectProperty` — links to a *resource*
- `owl:DatatypeProperty` — links to a *literal*

The glue

- `rdfs:subClassOf` / `rdfs:subPropertyOf`
- `rdfs:domain` / `rdfs:range`

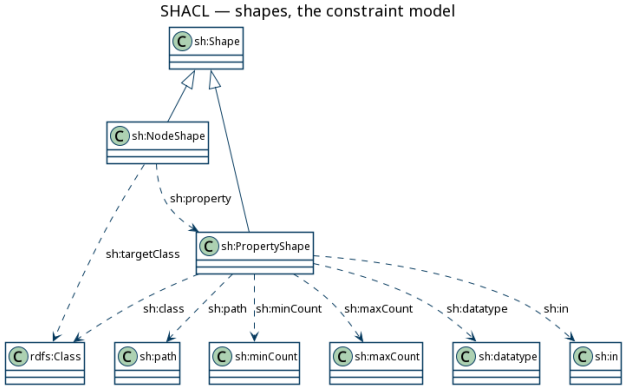
Open world: nothing is *forbidden* unless you say so.

uuid ea

©uuidea cc by-sa 4.0

SHACL — the constraint model (L2)

Shapes: what a valid instance must look like



Shapes

- `sh:NodeShape` — constraints on a *whole node*
- `sh:PropertyShape` — constraints on *one property*

Targeting

- `sh:targetClass` — which class the shape applies to

Property-shape constraints

- `sh:in` — a closed value set
- `sh:datatype` — the value type
- `sh:minCount` / `sh:maxCount` — cardinality
- `sh:path` — which property

Closed world: what is missing *means* something here.

Challenges for integrating information standards

- Information standards are modelled according to different conventions
  - CIM, INSPIRE → UML
  - openADR (OpenAPI) → OpenAPI
  - prov-o → OWL

uuid ea

©uuidea cc by-sa 4.0

Challenges for integrating information standards

- Information standards are modelled according to different conventions
  - CIM, INSPIRE → UML
  - openADR (OpenAPI) → OpenAPI
  - prov-o → OWL
- Information standards use different terms to define the same concept
- Information standards use the same terms to mean different things

uuid ea

©uuidea cc by-sa 4.0

Challenges for integrating information standards

- Information standards are modelled according to different conventions
  - CIM, INSPIRE → UML
  - openADR (OpenAPI) → OpenAPI
  - prov-o → OWL
- Information standards use different terms to define the same concept
- Information standards use the same terms to mean different things
- Information standards are defined from the perspective of a specific domain and are not equally specific about every concept
  - CIM doesn't think of Addresses as identifiable objects
  - INSPIRE thinks of a Substation primarily as a point on a map (it models it as an 'Appurtenance', as it does switches, transformers etc), you have to look in the AppurtenanceType enumeration to figure out what the object represents
  - INSPIRE makes no distinction between Asset and PowerSystemResources

uuid ea

©uuidea cc by-sa 4.0

What a well-defined information standard should be able to do

Req. 1

An information standard must serve multiple use cases

uuid ea

©uuidea cc by-sa 4.0

What a well-defined information standard should be able to do

Req. 1

An information standard must serve multiple use cases

Req. 2

An information standard must not favor specific implementation technologies

uuid ea

©uuidea cc by-sa 4.0

What a well-defined information standard should be able to do

Req. 1

An information standard must serve multiple use cases

Req. 2

An information standard must not favor specific implementation technologies

Req. 3

An information standard must not contain logical inconsistencies (definitions of empty sets)

uuid
ea

©uuidea cc by-sa 4.0

Roadmap: how the rest of today addresses the requirements

One requirement at a time — then an *in practice* act applying them all

Requirement	Addressed by
Req. 1 — serve multiple use cases	Kinds of information models: separating *vocabulary* from *constraint*; later: profiles as use-case-specific L2 models
Req. 2 — technology-agnostic	MOF: metamodels, expressivity, choosing a *pivot model*, metamodel transformations; later: generated artifacts (Avro, JSON Schema, SQL)
Req. 3 — no logical inconsistencies	Upper ontologies & ontology alignment: spotting and fixing semantic conflicts (the `cim:Outage` example)

The goal

A practical guide to how to meet all these requirements. Looking at how Netbeheer Nederland addresses them. And finally looking back at openADR.

©uuidea cc by-sa 4.0

Requirement 1: serving multiple use cases

An information standard must serve multiple use cases

uuid ea

©uuidea cc by-sa 4.0

Kinds of Models

Natural and formal languages

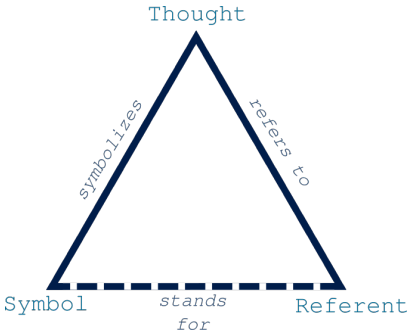


Figure: Semantic Triangle

uuid ea

 ©uuidea cc by-sa 4.0

Kinds of Models

Natural and formal languages

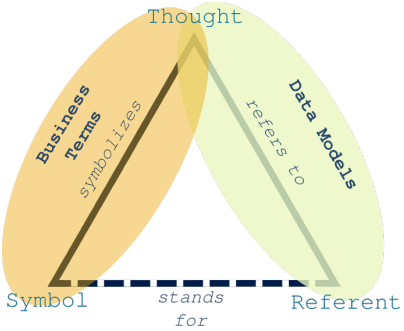


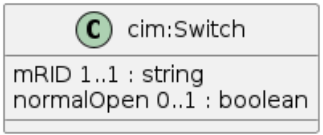
Figure: Semantic Triangle

uuid ea

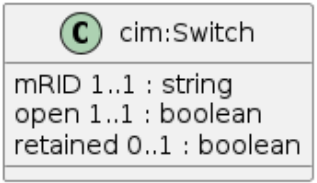
©uuidea cc by-sa 4.0

Kinds of Models

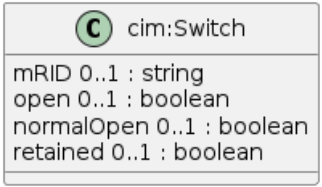
Vocabularies and Constraints



a Switch class with
2 attributes
some compulsory



a Switch class with
2 different attributes
some compulsory



a Switch class with
all attributes
all optional

uuid
ea

©uuidea cc by-sa 4.0

Kinds of Models

Vocabularies and constraints, definitions

Definition 6

Vocabulary Model : Models the reusable terms of the things the model describes using the open world assumption. This is the part of an information model that is reusable across use cases

Definition 7

Constraint Model : Models the data structure of the things the model describes using the closed world assumption. This part of an information model is use case specific

uuid ea

©uuidea cc by-sa 4.0

Kinds Of Models

A rough taxonomy of models

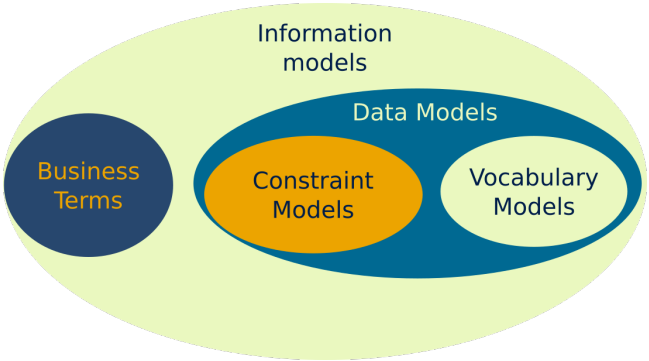


Figure: Venn diagram of model types

uuid ea

 ©uuidea cc by-sa 4.0

Addressing requirement 1

- An information standard must serve multiple use cases (Req. 1), which requires:
  - Separation of vocabulary and constraint models
  - Focussing on vocabulary model integration
  - Using the Open World Assumption for vocabulary integration

uuid ea

©uuidea cc by-sa 4.0

Requirement 2: being information technology-agnostic

An information standard must not favor specific implementation technologies

uuid ea

©uuidea cc by-sa 4.0

Models of Models

meta-meta-models	MOF, RDF(S)	
meta-models	UML, RDFS/OWL, SHACL, SQL, JSON-schema, Avro Schema,XSD, ecore	
models	IEC-CIM, 61850, HR-XML, prov-o, ssn, qudt, dcat dqv, INSPIRE, nen-2660, BFO, IFC...	CIM-EQ, CIM-SSH, CIM-SV, DCAT-AP, GML...
instance data	substations, employees, events, meters etc.	

uuid ea

©uuidea cc by-sa 4.0

Meta-model Expressivity

Construct	Description
data structure	how data about a thing is structured
set (class)	a set of things
individual	a single thing
role	properties, attributes, relationships
datatype	ways to format literals
set membership	ability to say an individual belongs to a set
set inclusion	subclass/superclass relationships
set equivalence	ability to say 2 sets are actually the same
set disjunctiveness	ability to say 2 sets share no individuals
cardinality restrictions	ability to say how often a role may occur

uuid ea

©uuidea cc by-sa 4.0

Comparing expressivity

	Open World Assumption			Closed World Assumption				
Construct	**RDFS/OWL**	**SHACL**	**LinkML**	**UML**	**JSON Schema****	**Apache Avro**	**SQL**	**XSD**
data structure		x	x	x	x	x	x	x
set (class)	x		kinda...					
individual	x	x	x	x	x	x	x	x
role	x	x	x	x	x	x	x	x
datatype	x	x	x	x	x	x	x	x
set membership	x			x			x	x
set inclusion	x	kinda...	kinda...	kinda...	kinda...		x	kinda...
set equivalence	x							
set disjunctiveness	x			implied	implied	implied	implied	implied
cardinality restrictions	x	x	x	x	x	limited	limited	x

uuid ea

©uuidea cc by-sa 4.0

Towards technology-agnostic information standards

Choosing a pivot model

	Open World Assumption	Closed World Assumption						
Construct	**RDFS/OWL**	**SHACL**	**LinkML**	**UML**	**JSON Schema**	**Apache Avro**	**SQL**	**XSD**
data structure		x	x	x	x	x	x	x
set (class)	x		kinda...					
individual	x	x	x	x	x	x	x	x
role	x	x	x	x	x	x	x	x
datatype	x	x	x	x	x	x	x	x
set membership	x			x			x	x
set inclusion	x	kinda...	kinda...	kinda...	kinda...		x	kinda...
set equivalence	x							
set disjunctiveness	x			implied	implied	implied	implied	implied
cardinality restrictions	x	x	x	x	x	limited	limited	x

uuid ea

©uuidea cc by-sa 4.0

Towards technology-agnostic information standards

Meta-model transformations

	Open World Assumption		Closed World Assumption					
Construct	**RDFS/OWL**	**SHACL**	**LinkML**	**UML(xmi)**	**JSON Schema**	**Apache Avro**	**SQL**	**XSD**
data structure		Class	Class	Class	object	record	table	complexType
set (class)	Class		class_uri					
individual	Resource	Resource	permissible_values	Instance	instance	implied	record	complex element
role	Property	PropertyShape	Slot	Property	property	field	column	attribute
datatype	DataType	DatatypeConstraintComponent	Type	Primitve	basic type	Primitive Type	data type	Datatype
set membership	type			type			implied	implied
set inclusion	subClassOf	and***	is_a	Generalization***	allOf***		union	extension***
set equivalence	EquivalentClass							
set disjunctiveness	disjointWith			implied	implied	implied	implied	implied
cardinality restrictions	min/maxCardinality	min/maxCount	minimum/maximum_cardinality	lower/upperValue	min/maxItems	limited	limited	min/maxOccurs

uuid ea

©uuidea cc by-sa 4.0

Addressing requirement 2

- An information standard must not favor specific implementation technologies (Req. 2), which requires:
  - An understanding of models, meta-models, and in some cases meta-meta-models
  - The ability to transform both vocabulary models as well as constraint models between technologies (and their respective meta-models)
  - The selection of a pivot model that has sufficient expressivity to allow transformation to all (most) other meta-models

uuid ea

©uuidea cc by-sa 4.0

Some Definitions

Definition 8

Closed World Assumption (CWA): *Information that is unknown is presumed to be false*

Definition 9

Open World Assumption(OWA): *Information that is unknown is presumed neither true nor false*

Definition 10

Unique Naming Assumption(UNA): *Different names are presumed to refer to different things in reality (this is a specific consequence of the CWA)*

uuid ea

©uuidea cc by-sa 4.0

Avoiding Inconsistencies

Some definitions

Definition 11

Universe of Discourse: *The set of entities over which certain variables of interest in some formal treatment may range*

Definition 12

Modelling adequacy: *The degree to which a model is capable of describing relevant phenomena. The accuracy with which the variables in the model describe the entities in the real world*

Definition 13

Interpretation: *An assignment of meaning to the symbols of a formal language*

Definition 14

Interpreter's expectation of a model: *The variables that an interpreter (domain expert) expects to be in the Universe of Discourse*

©uuidea cc by-sa 4.0

Requirement 3: being consistent

An information standard must not contain logical inconsistencies (definitions of empty sets)

uuid ea

©uuidea cc by-sa 4.0

Avoiding Inconsistencies

Example

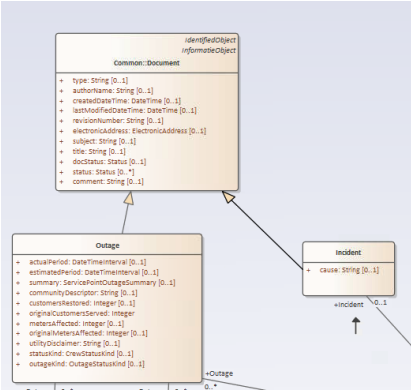


Figure: Events or States modelled as Documents

uuid ea

 ©uuidea cc by-sa 4.0

Using Upper Ontologies

Intro and benefits

- Upper ontologies provide a high over classification framework that can form the basis for interpreting models describing different Universes of Discourse
- Upper ontologies can make integrating information standards a lot easier and more formal
- Upper ontologies make basic assumptions about our reality explicit
- Two relevant examples
  - Basic Formal Ontology (BFO): well known in the scientific Linked Data community
  - nen-2660: the Upper Ontology for the Dutch construction industry. A simplified version of the BFO

uuid ea

©uuidea cc by-sa 4.0

Using Upper Ontologies

Limitations and pitfalls

- Upper ontologies enforce a specific perspective on the world
  - This creates barriers for underrepresented perspectives of what, for example, the energy system 'is' and what its existence (or lack thereof) implies for people or other things with intrinsic value, and representing those perspectives in data
  - This could be a criticism of any applied formal language (and therefore all structured data)

uuid ea

©uuidea cc by-sa 4.0

Using Upper Ontologies

Limitations and pitfalls

- Upper ontologies enforce a specific perspective on the world
  - This creates barriers for underrepresented perspectives of what, for example, the energy system 'is' and what its existence (or lack thereof) implies for people or other things with intrinsic value, and representing those perspectives in data
  - This could be a criticism of any applied formal language (and therefore all structured data)
- Upper ontologies can get very intangible, causing domain experts to be unable to validate a model
  - Explain to an asset manager that their inspection report is actually also a "generically dependent continuant" (BFO)...

uuid ea

©uuidea cc by-sa 4.0

Using Upper Ontologies

Limitations and pitfalls

- Upper ontologies enforce a specific perspective on the world
  - This creates barriers for underrepresented perspectives of what, for example, the energy system 'is' and what its existence (or lack thereof) implies for people or other things with intrinsic value, and representing those perspectives in data
  - This could be a criticism of any applied formal language (and therefore all structured data)
- Upper ontologies can get very intangible, causing domain experts to be unable to validate a model
  - Explain to an asset manager that their inspection report is actually also a "generically dependent continuant" (BFO)...
- Upper ontologies are often less formal than they seem to suggest
  - nen-2660 considers an Organization to be a Physical Object
  - I'm still on the fence whether an 'upper ontology' is the way to go, or if 'upper business terms' are a more accurate representation

uuid ea

©uuidea cc by-sa 4.0

Using Upper Ontologies

Example (my personal favorite)

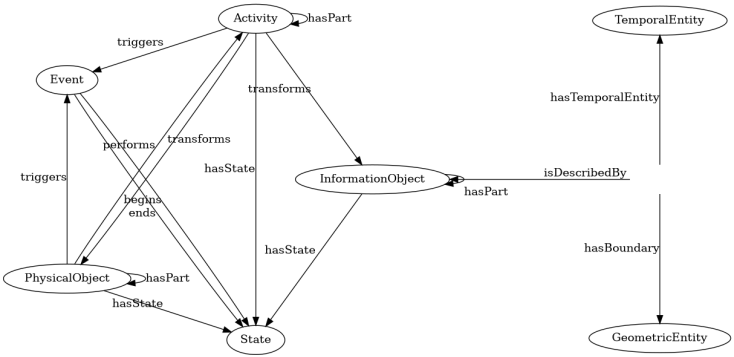


Figure: Upper ontology suggested by nen-2660

uuid ea

 ©uuidea cc by-sa 4.0

Using upper ontologies

Fixing a cim:Outage (according to nen-2660)

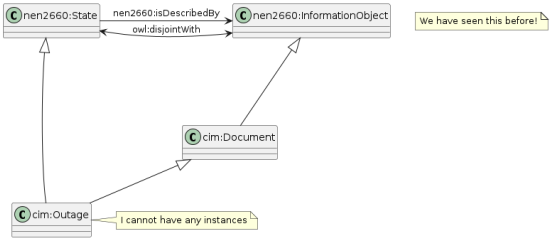


Figure: Outage as State *and* InformationObject

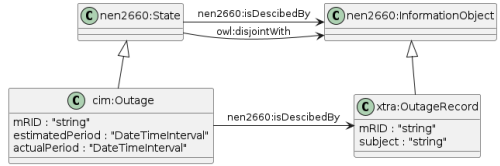


Figure: Separating the state from the record that describes it

uuid ea

 ©uuidea cc by-sa 4.0

Addressing requirement 3

- An information standard must not contain logical inconsistencies (definitions of empty sets) (Req. 3), which requires:
  - Semantic harmonization of the standards
  - The selection of an upper ontology helps identify potential inconsistencies

uuid ea

©uuidea cc by-sa 4.0

A real example: LV measurement events

Low-voltage measurement values published on a Kafka topic.

- The current schema: an Avro file.
- The same information is also stored in a warehouse (Postgres) and exposed via an HTTP query API
- Today these three exist side by side, maintained *by hand*, and they occasionally disagree

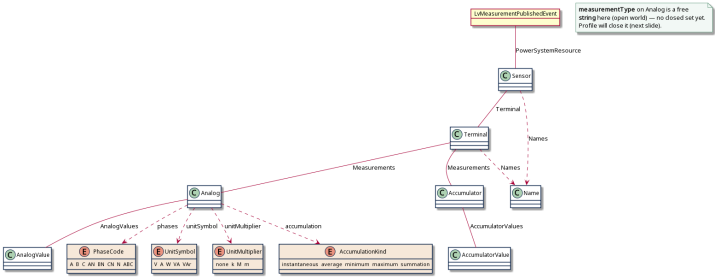
We will trace one event through L1, L2, L3

One event $\approx$ *one sensor, one terminal, several measurements, several values*.

uuid ea

©uuidea cc by-sa 4.0

L1 — the measurement information model



Everything here is standard or convention:

- `Sensor, Terminal, Analog, Accumulator, AnalogValue` — CIM core classes (IEC 61970-301)
- `PhaseCode, UnitSymbol, UnitMultiplier` — CIM enumerations
- `mRID` — IEC 61970-552 identifier convention
- `QualityMetadata` — W3C DQV, re-used

Nothing local, nothing proprietary. This is what we get for free — the definitions our engineers currently re-type into every schema file.

uuid ea

©uuidea cc by-sa 4.0

L2 — Profile (closed world)

A profile *references* an information model and *adds*:

- **scope constraints** — which parts of the Ontology are used
- **cardinalities** — required / optional / exactly-one
- **value-space** — closed enums, pattern, range

Closed world

No statement is made if not in-scope. Absence *means* something here.

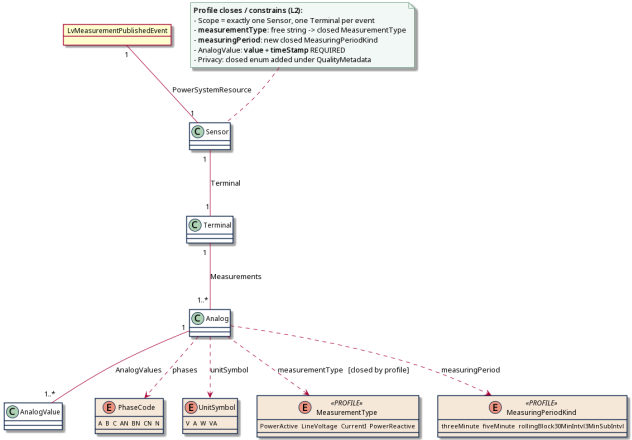
syntax agnostic

We are just describing what the data looks like, so it can be expressed in any syntax we might need, insuring consistent semantics across syntaxes.

uuid ea

©uuidea cc by-sa 4.0

L2 — the exchange profile



What the profile adds over the IM (constraints only):

- two L1 enums *closed* to the values of this exchange: `MeasuringPeriodKind` and `Privacy`
- `measurementType` closed to: `PowerActive`, `LineVoltage`, `CurrentI`, ...
- required fields: every `AnalogValue` *must* have `timeStamp` + `value`
- cardinalities: exactly one `Sensor`, one `Terminal` per event

Still no format anywhere

This is the *only* place those decisions are written down — not in three schema files, not in a wiki page.

uuid ea

©uuidea cc by-sa 4.0

Artifact 1/3 — Avro (streaming, Kafka)

```
{
  "name" : "Analog",
  "type" : "record",
  "namespace" : "com.grid.lvmeas",
  "fields" : [
    { "name" : "mRID", "type" : [ "null", "string" ] },
    { "name" : "phases",
      "type" : { "type" : "enum", "name" : "PhaseCode",
                 "symbols" : [ "A","B","C","AN","BN","CN","N" ] } },
    { "name" : "unitSymbol",
      "type" : { "type" : "enum", "name" : "UnitSymbol",
                 "symbols" : [ "V","A","VA","W" ] } },
    { "name" : "AnalogValues",
      "type" : { "type" : "array", "items" : "AnalogValue" } }
  ]
}
```

uuid ea

©uuidea cc by-sa 4.0

Artifact 2/3 — JSON Schema (request/response HTTP)

```
{
  "$id" : "https://grid.example/schemas/analog-measurement",
  "type" : "object",
  "required" : [ "mRID", "AnalogValues" ], /* from the PROFILE */
  "properties" : {
    "mRID" : { "$ref" : ".../schemas/mRID" },
    "phases" : { "enum" : [ "A","B","C","AN","BN","CN","N" ] },
    "unitSymbol" : { "enum" : [ "V","A","VA","W" ] },
    "AnalogValues" : { "type" : "array", "items" : {
      "required" : [ "timeStamp", "value" ], /* from the PROFILE */
      "properties" : {
        "timeStamp" : { "type" : "integer" },
        "value" : { "type" : "number" }
      }
    } }
  }
}
```

uuid ea

©uuidea cc by-sa 4.0

Artifact 3/3 — SQL DDL (batch / warehouse)

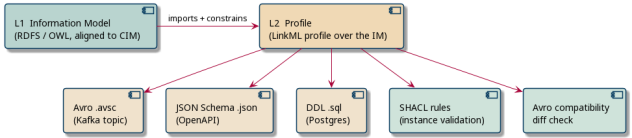
```
CREATE TABLE analog_measurement (
    mRID          VARCHAR(64) PRIMARY KEY,
    phases        VARCHAR(4)  NOT NULL,
    unit_symbol   VARCHAR(8)  NOT NULL,
    created_at    TIMESTAMP   NOT NULL
);

CREATE TABLE analog_value (
    mRID            VARCHAR(64) NOT NULL,
    time_stamp      BIGINT      NOT NULL,      /* epoch millis: same instant
        as Avro */
    value           NUMERIC(12,4),
    FOREIGN KEY (mRID) REFERENCES analog_measurement(mRID)
);
```

uuid ea

©uuidea cc by-sa 4.0

Toolchain



Consistency comes from the graph structure, not from discipline:

- every artifact is *downstream* of the profile
- a change to `PhaseCode` happens *once* in the profile
- the validator is derived *from* the same profile — one rule set
- review becomes: does this change the profile? what does it do to each artifact?

uuid ea

©uuidea cc by-sa 4.0

The toolchain

- **L1** **RDFS/OWL** — an aligned vocabulary (here CIM IEC 61970-301, with `owl:equivalentClass`)
- **L2** a **LinkML profile** that imports the IM and narrows it ⇒ `linkml-shacl` emits the conformance rules
- **L3** `linkml-avro`, `linkml-jsonschema`, `linkml-sql` — one command each, all from *the same* profile

```
# L1  information model: hand-authored RDFS/OWL (aligned to CIM IEC 61970-301)
# L2  profile    : LinkML profile over the IM
# L3  artifacts : generated from that one profile

$ linkml-shacl      lvprofile.yaml -o lvprofile-shacl.ttl      # validator
$ linkml-avro       lvprofile.yaml -o lvmeas.avsc              # Kafka
$ linkml-jsonschema lvprofile.yaml -o schemas/analog.json      # OpenAPI
$ linkml-sql        lvprofile.yaml --dialect postgres -o ddl/pg.sql
$ shacl-validate    lvprofile-shacl.ttl instance.ttl
```

One profile. Three artifacts. One validator.

All of them *generated* in the same CI job; none of them hand-edited.

uuid ea

©uuidea cc by-sa 4.0

Data Uitwisseling

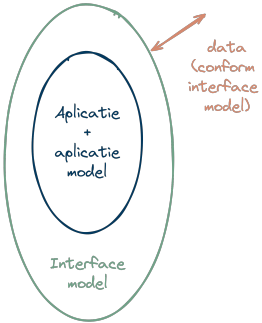


Figure: Data Uitwisseling zonder je applicatie overhoop te gooien

uuid
ea

©uuidea cc by-sa 4.0

Data Uitwisseling

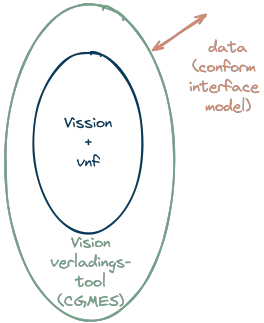


Figure: Data Uitwisseling zonder je applicatie overhoop te gooien

uuid
ea

©uuidea cc by-sa 4.0

CIM vs CGMES

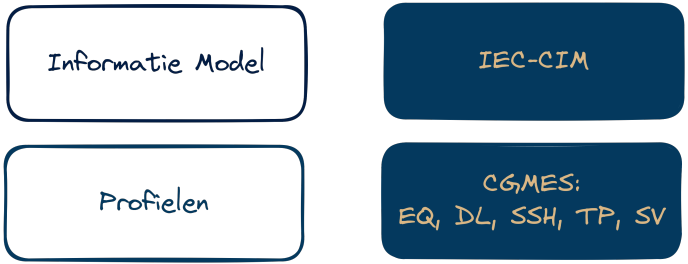


Figure: Informatiemodellen en profielen

uuid ea

©uuidea cc by-sa 4.0

Hoe de profielen samenhangen

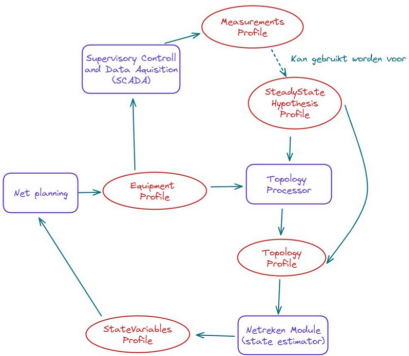


Figure: Informatiemodellen en profielen

uuid ea

©uuidea cc by-sa 4.0

How Netbeheer Nederland runs this in practice

Observations, not advice — check with your peers

- Netbeheer Nederland publishes **CGMES-based profiles** as the exchange language for grid data between DSOs and the TSO:
  - documentation: modellen.netbeheernederland.nl/cgmes
  - built on IEC 61970 CIM (the vocabulary) + ENTSO-E CGMES (the Dutch/European profile)

uuid ea

©uuidea cc by-sa 4.0

How Netbeheer Nederland runs this in practice

Observations, not advice — check with your peers

- Netbeheer Nederland publishes **CGMES-based profiles** as the exchange language for grid data between DSOs and the TSO:
  - documentation: modellen.netbeheernederland.nl/cgmes
  - built on IEC 61970 CIM (the vocabulary) + ENTSO-E CGMES (the Dutch/European profile)
- The profile is *use-case specific*: congestion calculation, capacity calculation, topology exchange — each is a closed-world subset of the open-world model.

uuid ea

©uuidea cc by-sa 4.0

How Netbeheer Nederland runs this in practice

Observations, not advice — check with your peers

- Netbeheer Nederland publishes **CGMES-based profiles** as the exchange language for grid data between DSOs and the TSO:
  - documentation: modellen.netbeheernederland.nl/cgmes
  - built on IEC 61970 CIM (the vocabulary) + ENTSO-E CGMES (the Dutch/European profile)
- The profile is *use-case specific*: congestion calculation, capacity calculation, topology exchange — each is a closed-world subset of the open-world model.
- The schemas (GML/RDF/XML artifacts) are *derived* from the profile, not the other way round — same discipline as in our four-layer stack.

uuid ea

©uuidea cc by-sa 4.0

For DSOs in the Netherlands: the practical consequence

- your exchange *must* be expressible in CIM terms, because the other side (TSO, other DSOs, market parties) is already there
- DER assets that want to be *seen* in that exchange need a CIM representation — that is the gap this week closes.

Exercise (optional, in pairs)

List the artifacts your openADR flow already produces and mark which already have a CIM counterpart — and which don't.

uuid ea

©uuidea cc by-sa 4.0

The standard as defined — what the yardstick applies to

- The **normative reference of OADR 3 is an OpenAPI document (YAML)**: the Alliance calls it "the normative reference for OADR 3", and it "supersedes any statements made in other documentation".
- The accompanying **"Definitions" document** restates the same model in prose — and is, by that very clause, *not* the normative one.
- What the standard defines is a **REST + JSON API**: paths, operations, and the payload schemas that move over it. Its predecessor (2.0b, SOAP/XML) has to be implemented alongside it.

uuid ea

©uuidea cc by-sa 4.0

Instance data has no type

The validator checks *structure* — not which thing a thing is

What the validator has — one schema

```
{
  "type" : "object",
  "required" : [ "mRID", "name", "nominalVoltage" ],
  "properties" : {
    "mRID" : { "type" : "string" },
    "name" : { "type" : "string" },
    "nominalVoltage" : { "type" : "number" }
  }
}
```

What arrives in the exchange — two instances

```
[
  {
    "mRID" : "US.EPE:SUB.104200",
    "name" : "Northgate 138/13 kV",
    "nominalVoltage" : 138000
  },
  {
    "mRID" : "US.EPE:TRF.0077",
    "name" : "TG-3",
    "nominalVoltage" : 138000
  }
]
```

The first is a *Substation*; the second a *Transformer*. The payload never says which — only the sender knows.

The verdict: both instances validate ✓ — including the Transformer, where a Substation was expected. The validator can only check that the structure matches, so it cannot decide which *type* the

©uuidea cc by-sa 4.0

openADR 3.0 against the three requirements

The requirements as defined earlier in this deck — the standard as defined

Requirement	openADR 3.0
Req. 1	
serves multiple use cases	Use cases enter as **profiles** — slices of the API surface — and certification is **per profile**. The content shared between use cases is the payload schemas of the normative YAML: shared *as an artifact*, not as concepts that can be referenced or imported on their own. *This seems to work!*
Req. 2	
not technology-specific	OADR 3 is defined as a **REST + JSON API**; the model content sits inside that description of the transport. The predecessor already marks the pattern: 2.0b (SOAP/XML) and 3.0 (REST/JSON) are two serializations for the same subject matter, each with its own normative description. So openADRm in is current form, is not technology agnostic.
Req. 3	
no empty sets	JSON schema classes do not define mathematical sets, *this is cannot be validated.*

The table describes the standard against the requirements. It makes no proposal and offers no remediation.

©uuidea cc by-sa 4.0

Takeaways — Day 1

1. **Defining an information standard**
   - interoperability is a community quality; a standard is what they decide to make *testable*

uuid
ea

©uuidea cc by-sa 4.0

Takeaways — Day 1

1. **Defining an information standard**
   - interoperability is a community quality; a standard is what they decide to make *testable*
2. **The best practice stack**
   - L0 concepts (SKOS) → L1 open-world vocabulary → L2 profile → L3 *derived* artifacts; schema ≠ meaning
   - MOF: choose a pivot model, don't chase one; validate from the profile (SHACL)
   - proven in practice: Netbeheer Nederland runs this with CIM/CGMES

uuid ea

©uuidea cc by-sa 4.0

Takeaways — Day 1

1. **Defining an information standard**
   - interoperability is a community quality; a standard is what they decide to make *testable*
2. **The best practice stack**
   - L0 concepts (SKOS) → L1 open-world vocabulary → L2 profile → L3 *derived* artifacts; schema ≠ meaning
   - MOF: choose a pivot model, don't chase one; validate from the profile (SHACL)
   - proven in practice: Netbeheer Nederland runs this with CIM/CGMES
3. **openADR 3.0 through that lens**
   - the normative core of the standard is a *serialization*: the OpenAPI YAML, with the model defined inside it
   - the three requirements of the day attach to exactly that choice: multiple use cases, technology-independence, no empty sets

uuid ea

©uuidea cc by-sa 4.0

Conclusion

- A well-defined information standard facilitates both organization-wide and inter-organization data management
  - It allows the use of expert models for *every* domain
  - Standardization of information helps with every aspect of (FAIR) data management
- Defining and standardizing information is about standardizing standards
  - The motivations are identical
  - The methods are identical
  - Just one level of abstraction higher
- A well-defined information standard requires
  - A clear distinction between vocabulary models and constraint models (req. 1)
  - Harmonized metamodels; choosing an appropriate pivot model is key (req. 2)
  - A fair amount of interpretation, but an upper ontology can provide a structure for interpretation (req. 3)
- Applied to openADR 3.0: what the standard makes normative is the *serialization* of the model — the OpenAPI document — and the requirements of the day apply to that artifact directly

uuid ea

©uuidea cc by-sa 4.0

Day 2: CIM, IEC 61850, the DSO and the DER

- CIM and 61850 in the SGAM context — and what that relationship teaches openADR
- The origins of CIM
- CIM and *netrekenen*: why the grid is calculated, not just managed
- Kirchhoff's law, and the four things to exchange to be able to calculate:
  - Connectivity
  - Topology
  - State
  - Results
- What a **bedrijfsvoeringscentrum** must know about a charging point to be able to steer it well
- When CIM is interesting for openADR — and honestly, when it is not
- CIM: the good parts, the rough parts, and the pitfalls
- Object-oriented or semantic — and what that means for your models
- CIM and OpenAPI
- Standards to keep in your gate: SOSA/SSN, QU, and metadata standards

uuid ea

©uuidea cc by-sa 4.0

Thank you — questions?

joep@uuidea.eu

GitHub: github.com/uuidea

Content shared under CC BY-SA 4.0

uuid
ea